Experiment 1: Class Fundamentals.

Program 1.1: Student Information System

Problem Statement:

Develop a Java program to create a Student class with data members such as Roll Number, Name, Branch, and CGPA. Create an object of the class, initialize the data, and display the student information using a method.

```
public class Student {
    int rollNumber;
    String name;
    String branch;
    double cgpa;

    void display() {
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Name: " + name);
        System.out.println("Branch: " + branch);
        System.out.println("CGPA: " + cgpa);
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        s1.rollNumber = 101;
        s1.name = "Aditi Sharma";
        s1.branch = "Computer Science";
        s1.cgpa = 9.12;

        s1.display();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Student.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Student
Roll Number: 101
Name: Aditi Sharma
Branch: Computer Science
CGPA: 9.12
```

Program 1.2: Employee Information System

Problem Statement:

Develop a Java program to create an Employee class containing Employee ID, Name, Department, and Salary. Create an object of the class and display the employee details.

```
public class Employee {
    int empId;
    String name;
    String department;
    double salary;

    void display() {
        System.out.println("Employee ID: " + empId);
        System.out.println("Name: " + name);
        System.out.println("Department: " + department);
        System.out.println("Salary: " + salary);
    }

    public static void main(String[] args) {
        Employee e1 = new Employee();
        e1.empId = 501;
        e1.name = "Rohan Verma";
        e1.department = "Human Resources";
        e1.salary = 45000.0;

        e1.display();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Employee.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Employee
Employee ID: 501
Name: Rohan Verma
Department: Human Resources
Salary: 45000.0
```

Experiment 2: Declaring Objects

Program 2.1: Book Details

Problem Statement:

Write a Java program to declare and create three objects of a Book class. Display the details of all books using object ref

```
public class Book {
    String title;
    String author;
    double price;

    void display() {
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Price: " + price);
        System.out.println("----------------------");
    }

    public static void main(String[] args) {
        Book b1 = new Book();
        b1.title = "Java Programming";
        b1.author = "James Gosling";
        b1.price = 599.0;

        Book b2 = new Book();
        b2.title = "The Alchemist";
        b2.author = "Paulo Coelho";
        b2.price = 299.0;

        Book b3 = new Book();
        b3.title = "Wings of Fire";
        b3.author = "A.P.J. Abdul Kalam";
        b3.price = 350.0;

        b1.display();
        b2.display();
        b3.display();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Book.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Book
Title: Java Programming
Author: James Gosling
Price: 599.0
----------------------
Title: The Alchemist
Author: Paulo Coelho
Price: 299.0
----------------------
Title: Wings of Fire
Author: A.P.J. Abdul Kalam
Price: 350.0
----------------------
```

Program 2.2: Array of Student Objects

Problem Statement:

Write a Java program to create an array of five Student objects, accept their details from the user, and display all student information

```java
import java.util.Scanner;

public class Info {
    int rollNumber;
    String name;
    double marks;

    void display() {
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Name: " + name);
        System.out.println("Marks: " + marks);
        System.out.println("-----------------------");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Info[] students = new Info[5];

        for (int i = 0; i < 5; i++) {
            students[i] = new Info();
            System.out.println("Enter Roll Number, Name, Marks:");
            students[i].rollNumber = sc.nextInt();
            students[i].name = sc.next();
            students[i].marks = sc.nextDouble();
        }

        for (int i = 0; i < 5; i++) {
            students[i].display();
        }
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Info.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Info
Enter Roll Number, Name, Marks:
98
Prazod
100
Enter Roll Number, Name, Marks:
97
Trinadh
100
Enter Roll Number, Name, Marks:
22
Sai
100
Enter Roll Number, Name, Marks:
24
Navadeep
100
Enter Roll Number, Name, Marks:
25
Avinash
101
Roll Number: 98
Name: Prazod
Marks: 100.0
-----------------------
Roll Number: 97
Name: Trinadh
Marks: 100.0
-----------------------
Roll Number: 22
Name: Sai
Marks: 100.0
-----------------------
Roll Number: 24
Name: Navadeep
Marks: 100.0
-----------------------
Roll Number: 25
Name: Avinash
Marks: 101.0
-----------------------
```

Experiment 3: Assigning Object Reference Variables

Program 3.1: Reference Assignment

Problem Statement:

Develop a Java program to create a Student object and assign its reference to another reference variable. Modify the object through one reference and observe the changes using the other reference.

```
public class Details {
    int rollNumber;
    String name;
    double marks;

    void display() {
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Name: " + name);
        System.out.println("Marks: " + marks);
        System.out.println("----------------------");
    }

    public static void main(String[] args) {
        Details d1 = new Details();
        d1.rollNumber = 101;
        d1.name = "Aditi Sharma";
        d1.marks = 89.5;

        Details d2 = d1; // d2 now refers to the same object as d1

        System.out.println("Before modification (via d1):");
        d1.display();

        System.out.println("Before modification (via d2):");
        d2.display();

        // Modify the object through d2
        d2.name = "Rohan Verma";
        d2.marks = 95.0;

        System.out.println("After modifying through d2:");

        System.out.println("d1 now shows:");
        d1.display();

        System.out.println("d2 now shows:");
        d2.display();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Details.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Details
Before modification (via d1):
Roll Number: 101
Name: Aditi Sharma
Marks: 89.5
----------------------
Before modification (via d2):
Roll Number: 101
Name: Aditi Sharma
Marks: 89.5
----------------------
After modifying through d2:
d1 now shows:
Roll Number: 101
Name: Rohan Verma
Marks: 95.0
----------------------
d2 now shows:
Roll Number: 101
Name: Rohan Verma
Marks: 95.0
----------------------
```

Program 3.2: Comparing Object References

Problem Statement:

Write a Java program to create two reference variables pointing to the same Employee object. Compare the references using the == operator and display the result.

```
public class Staff {
    int empId;
    String name;
    double salary;

    void display() {
        System.out.println("Employee ID: " + empId);
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
        System.out.println("-----------------------");
    }

    public static void main(String[] args) {
        Staff s1 = new Staff();
        s1.empId = 501;
        s1.name = "Rohan Verma";
        s1.salary = 45000.0;

        Staff s2 = s1; // s2 points to the same object as s1

        System.out.println("Details via s1:");
        s1.display();

        System.out.println("Details via s2:");
        s2.display();

        // Comparing references using ==
        if (s1 == s2) {
            System.out.println("s1 and s2 refer to the SAME object.");
        } else {
            System.out.println("s1 and s2 refer to DIFFERENT objects.");
        }
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Staff.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Staff
Details via s1:
Employee ID: 501
Name: Rohan Verma
Salary: 45000.0
-----------------------
Details via s2:
Employee ID: 501
Name: Rohan Verma
Salary: 45000.0
-----------------------
s1 and s2 refer to the SAME object.
```

Experiment 4: Introducing Methods

Program 4.1: Calculator Using Methods

Problem Statement:

Develop a Java program to create a Calculator class with methods to perform addition, subtraction, multiplication, and division of two numbers.

```java
public class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int subtract(int a, int b) {
        return a - b;
    }

    int multiply(int a, int b) {
        return a * b;
    }

    double divide(int a, int b) {
        if (b == 0) {
            System.out.println("Error: Division by zero is not allowed.");
            return 0;
        }
        return (double) a / b;
    }

    public static void main(String[] args) {
        Calculator calc = new Calculator();

        int num1 = 20, num2 = 4;

        System.out.println("Addition: " + calc.add(num1, num2));
        System.out.println("Subtraction: " + calc.subtract(num1, num2));
        System.out.println("Multiplication: " + calc.multiply(num1, num2));
        System.out.println("Division: " + calc.divide(num1, num2));
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Calculator.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Calculator
Addition: 24
Subtraction: 16
Multiplication: 80
Division: 5.0
```

Program 4.2: Rectangle Operations

Problem Statement:

Write a Java program to create a Rectangle class with methods to calculate and display the area and perimeter of a rectangle.

```
public class Rectangle {
    double length;
    double width;

    double calculateArea() {
        return length * width;
    }

    double calculatePerimeter() {
        return 2 * (length + width);
    }

    void display() {
        System.out.println("Length: " + length);
        System.out.println("Width: " + width);
        System.out.println("Area: " + calculateArea());
        System.out.println("Perimeter: " + calculatePerimeter());
    }

    public static void main(String[] args) {
        Rectangle rect = new Rectangle();
        rect.length = 10.5;
        rect.width = 5.0;

        rect.display();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Rectangle.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Rectangle
Length: 10.5
Width: 5.0
Area: 52.5
Perimeter: 31.0
```

Experiment 5: Constructors

Program 5.1: Student Constructors

Problem Statement:

Develop a Java program to demonstrate the use of default and parameterized constructors for initializing student information.

```
class sconstructor {
    int id;
    String name;

    // Default constructor
    sconstructor() {
        id = 0;
        name = "Unknown";
    }

    // Parameterized constructor
    sconstructor(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void show() {
        System.out.println("ID: " + id + ", Name: " + name);
    }

    public static void main(String[] args) {
        sconstructor s1 = new sconstructor();              // default constructor
        sconstructor s2 = new sconstructor(101, "Alice");  // parameterized constructor

        s1.show();
        s2.show();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac sconstructor.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java sconstructor
ID: 0, Name: Unknown
ID: 101, Name: Alice

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>
```

Program 5.2: Constructor Overloading

Problem Statement:

Write a Java program to demonstrate constructor overloading in a BankAccount class by creating objects using different constructors.

```
class BankAccount {
    int accountNumber;
    String accountHolder;
    double balance;

    // Constructor 1: No arguments (default)
    BankAccount() {
        accountNumber = 0;
        accountHolder = "Unknown";
        balance = 0.0;
    }

    // Constructor 2: With account number and holder name
    BankAccount(int accountNumber, String accountHolder) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        balance = 0.0;
    }

    // Constructor 3: With all details
    BankAccount(int accountNumber, String accountHolder, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = balance;
    }

    void show() {
        System.out.println("Account No: " + accountNumber +
                           ", Holder: " + accountHolder +
                           ", Balance: $" + balance);
    }

    public static void main(String[] args) {
        // Using default constructor
        BankAccount acc1 = new BankAccount();

        // Using constructor with 2 parameters
        BankAccount acc2 = new BankAccount(1001, "Alice");

        // Using constructor with 3 parameters
        BankAccount acc3 = new BankAccount(1002, "Bob", 5000.0);

        acc1.show();
        acc2.show();
        acc3.show();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac BankAccount.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java BankAccount
Account No: 0, Holder: Unknown, Balance: $0.0
Account No: 1001, Holder: Alice, Balance: $0.0
Account No: 1002, Holder: Bob, Balance: $5000.0

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>
```

Experiment 6: The ‘this’ Keyword.

Program 6.1: Using this Keyword

Problem Statement:

Develop a Java program to demonstrate the use of the this keyword to distinguish instance variables from local variables.

```
class thisuse {
    int id;
    String name;

    // Constructor using 'this' to refer to instance variables
    thisuse(int id, String name) {
        this.id = id;       // this.id refers to instance variable
        this.name = name;   // this.name refers to instance variable
    }

    // Method using 'this' to distinguish variables
    void setDetails(int id, String name) {
        this.id = id;       // instance variable = local variable (parameter)
        this.name = name;
    }

    void show() {
        System.out.println("ID: " + id + ", Name: " + name);
    }

    public static void main(String[] args) {
        thisuse s1 = new thisuse(101, "Alice");  // using constructor
        s1.show();

        thisuse s2 = new thisuse(0, "");         // temporary object
        s2.setDetails(102, "Bob");               // using method with 'this'
        s2.show();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac thisuse.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java thisuse
ID: 101, Name: Alice
ID: 102, Name: Bob
```

Program 6.2: Constructor Chaining

Problem Statement:

Write a Java program to demonstrate constructor chaining using the this() constructor in a Student class.

```
class chaining {
    int id;
    String name;
    String course;

    // Constructor 1: No arguments
    chaining() {
        this(0, "Unknown");  // calls Constructor 2
        System.out.println("Default constructor called");
    }

    // Constructor 2: With id and name
    chaining(int id, String name) {
        this(id, name, "Not Enrolled");  // calls Constructor 3
        System.out.println("Constructor with id and name called");
    }

    // Constructor 3: With all details (main constructor)
    chaining(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
        System.out.println("Constructor with all parameters called");
    }

    void show() {
        System.out.println("ID: " + id + ", Name: " + name + ", Course: " + course);
    }

    public static void main(String[] args) {
        chaining s1 = new chaining();              // chains through all constructors
        s1.show();

        System.out.println();

        chaining s2 = new chaining(101, "Alice");  // chains to Constructor 3
        s2.show();

        System.out.println();

        chaining s3 = new chaining(102, "Bob", "Mathematics");  // direct call
        s3.show();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac chaining.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java chaining
Constructor with all parameters called
Constructor with id and name called
Default constructor called
ID: 0, Name: Unknown, Course: Not Enrolled

Constructor with all parameters called
Constructor with id and name called
ID: 101, Name: Alice, Course: Not Enrolled

Constructor with all parameters called
ID: 102, Name: Bob, Course: Mathematics
```

Experiment 7: Garbage Collection

Program 7.1: Demonstrating Garbage Collection

Problem Statement:

Develop a Java program to create multiple objects, make them eligible for garbage collection, and invoke System.gc().

```
public class multiple {

    String objectName;

    // Constructor
    public multiple(String name) {
        this.objectName = name;
        System.out.println(this.objectName + " successfully created.");
    }

    public static void main(String[] args) {
        System.out.println("--- 1. Creating Objects ---");
        multiple obj1 = new multiple("Object 1");
        multiple obj2 = new multiple("Object 2");
        multiple obj3 = new multiple("Object 3");

        System.out.println("\n--- 2. Making objects eligible for GC ---");

        // Method A: Nullifying the reference
        obj1 = null;
        System.out.println("Object 1 reference set to null.");

        // Method B: Reassigning the reference
        obj2 = obj3;
        System.out.println("Object 2 reference reassigned to Object 3.");

        // Method C: Creating an anonymous object
        new multiple("Object 4 (Anonymous)");
        System.out.println("Object 4 created without a reference.");

        System.out.println("\n--- 3. Requesting Garbage Collection ---");

        // Calling System.gc() suggests the JVM run the Garbage Collector
        System.gc();

        System.out.println("System.gc() called! The JVM will silently clean up Objects 1, 2, and 4 in the background.");
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac multiple.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java multiple
--- 1. Creating Objects ---
Object 1 successfully created.
Object 2 successfully created.
Object 3 successfully created.

--- 2. Making objects eligible for GC ---
Object 1 reference set to null.
Object 2 reference reassigned to Object 3.
Object 4 (Anonymous) successfully created.
Object 4 created without a reference.

--- 3. Requesting Garbage Collection ---
System.gc() called! The JVM will silently clean up Objects 1, 2, and 4 in the background.
```

Program 7.2: Object Eligibility for Garbage Collection

Problem Statement:

Write a Java program to demonstrate different ways of making objects eligible for garbage collection, such as null assignment, reassignment, and anonymous objects.

```java
public class GarbageCollectionDemo {

    String objectName;

    // Constructor to assign a name upon creation
    public GarbageCollectionDemo(String name) {
        this.objectName = name;
        System.out.println(this.objectName + " successfully created.");
    }

    public static void main(String[] args) {
        System.out.println("=== Step 1: Object Creation ===");
        GarbageCollectionDemo obj1 = new GarbageCollectionDemo("Object 1");
        GarbageCollectionDemo obj2 = new GarbageCollectionDemo("Object 2");
        GarbageCollectionDemo obj3 = new GarbageCollectionDemo("Object 3");

        System.out.println("\n=== Step 2: Making Objects Eligible for GC ===");

        // Method 1: Null Assignment
        // By setting the reference to null, "Object 1" is left without an active reference.
        obj1 = null;
        System.out.println("-> Null Assignment: 'Object 1' reference set to null.");

        // Method 2: Reassignment
        // By pointing obj2 to obj3, the original "Object 2" is orphaned and loses its reference.
        obj2 = obj3;
        System.out.println("-> Reassignment: 'Object 2' reference repointed to 'Object 3'.");

        // Method 3: Anonymous Object
        // Created without assigning it to a variable, meaning it is eligible the moment it finishes initializing.
        new GarbageCollectionDemo("Object 4 (Anonymous)");
        System.out.println("-> Anonymous Object: 'Object 4' created without a variable reference.");

        System.out.println("\n=== Step 3: Requesting Garbage Collection ===");

        // Suggests that the JVM run the Garbage Collector
        System.gc();
        System.out.println("-> System.gc() invoked. Objects 1, 2, and 4 will be cleaned up in the background.");
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac GarbageCollectionDemo.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java GarbageCollectionDemo
=== Step 1: Object Creation ===
Object 1 successfully created.
Object 2 successfully created.
Object 3 successfully created.

=== Step 2: Making Objects Eligible for GC ===
-> Null Assignment: 'Object 1' reference set to null.
-> Reassignment: 'Object 2' reference repointed to 'Object 3'.
Object 4 (Anonymous) successfully created.
-> Anonymous Object: 'Object 4' created without a variable reference.

=== Step 3: Requesting Garbage Collection ===
-> System.gc() invoked. Objects 1, 2, and 4 will be cleaned up in the background.
```

Experiment 8 : Method Overloading

Program 8.1: Overloading Arithmetic Methods

Problem Statement:

Develop a Java program to overload the add() method for different parameter lists.

```java
public class Add {

    // 1. Method to add two integers
    public int add(int a, int b) {
        return a + b;
    }

    // 2. Overloaded method to add three integers (Different number of parameters)
    public int add(int a, int b, int c) {
        return a + b + c;
    }

    // 3. Overloaded method to add two doubles (Different data types)
    public double add(double a, double b) {
        return a + b;
    }

    // 4. Overloaded method to add an integer and a double (Different order of data types)
    public double add(int a, double b) {
        return a + b;
    }

    public static void main(String[] args) {
        // Instantiating the 'Add' class
        Add calc = new Add();

        System.out.println("=== Method Overloading Demonstration ===");

        // Calls add(int, int)
        int sum1 = calc.add(15, 25);
        System.out.println("Adding two integers (15 + 25): " + sum1);

        // Calls add(int, int, int)
        int sum2 = calc.add(10, 20, 30);
        System.out.println("Adding three integers (10 + 20 + 30): " + sum2);

        // Calls add(double, double)
        double sum3 = calc.add(12.5, 7.5);
        System.out.println("Adding two doubles (12.5 + 7.5): " + sum3);

        // Calls add(int, double)
        double sum4 = calc.add(10, 5.5);
        System.out.println("Adding an int and a double (10 + 5.5): " + sum4);
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Add.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Add
=== Method Overloading Demonstration ===
Adding two integers (15 + 25): 40
Adding three integers (10 + 20 + 30): 60
Adding two doubles (12.5 + 7.5): 20.0
Adding an int and a double (10 + 5.5): 15.5
```

Program 8.2: Overloading Area Methods

Problem Statement:

Write a Java program to overload the area() method to calculate the area of a circle, rectangle, and square.

```java
public class AreaCalculator {

    // 1. Overloaded method for the area of a circle
    // Uses a 'double' parameter for the radius
    public double area(double radius) {
        return Math.PI * radius * radius;
    }

    // 2. Overloaded method for the area of a square
    // Uses an 'int' parameter for the side to differentiate from the circle method
    public int area(int side) {
        return side * side;
    }

    // 3. Overloaded method for the area of a rectangle
    // Uses two parameters (length and width)
    public int area(int length, int width) {
        return length * width;
    }

    public static void main(String[] args) {
        AreaCalculator calc = new AreaCalculator();

        System.out.println("=== Area Calculations Demonstration ===");

        // Calls area(double) -> Circle
        double circleArea = calc.area(5.5);
        System.out.println("Area of the Circle (radius 5.5): " + circleArea);

        // Calls area(int) -> Square
        int squareArea = calc.area(7);
        System.out.println("Area of the Square (side 7): " + squareArea);

        // Calls area(int, int) -> Rectangle
        int rectangleArea = calc.area(8, 12);
        System.out.println("Area of the Rectangle (length 8, width 12): " + rectangleArea);
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac AreaCalculator.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java AreaCalculator
=== Area Calculations Demonstration ===
Area of the Circle (radius 5.5): 95.03317777109123
Area of the Square (side 7): 49
Area of the Rectangle (length 8, width 12): 96
```

Experiment 9: Using Objects as Parameters

Program 9.1: Passing Student Object

Problem Statement:

Develop a Java program to pass a Student object as a parameter to a method and display the student details.

```
class Student {
    String name;
    int rollNumber;
    String course;

    // Constructor to initialize the student details
    public Student(String name, int rollNumber, String course) {
        this.name = name;
        this.rollNumber = rollNumber;
        this.course = course;
    }
}


public class methoddisplay {

    // Method that accepts a Student object as a parameter
    public static void displayStudentDetails(Student student) {
        System.out.println("--- Student Details ---");
        System.out.println("Name        : " + student.name);
        System.out.println("Roll Number : " + student.rollNumber);
        System.out.println("Course      : " + student.course);
    }

    public static void main(String[] args) {
        // 1. Create Student objects
     30    Student student1 = new Student("Alice Johnson", 101, "Computer Science");
        Student student2 = new Student("Bob Smith", 102, "Information Technology");

        // 2. Pass the objects to the display method
        displayStudentDetails(student1);

        System.out.println(); // Print a blank line for readability

        displayStudentDetails(student2);
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac methoddisplay.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java methoddisplay
--- Student Details ---
Name        : Alice Johnson
Roll Number : 101
Course      : Computer Science

--- Student Details ---
Name        : Bob Smith
Roll Number : 102
Course      : Information Technology
```

Program 9.2: Comparing Employee Salaries

Problem Statement:

Write a Java program to pass two Employee objects to a method that compares their salaries and displays the employee with the higher salary.

```java
// Define the Employee class using encapsulation
class Employee {
    private String name;
    private double salary;

    // Constructor to initialize the employee details
    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    // Getter method for the employee's name
    public String getName() {
        return name;
    }

    // Getter method for the employee's salary
    public double getSalary() {
        return salary;
    }
}

// Main class renamed to 'employeeobjects'
public class employeeobjects {

    // Method that accepts two Employee objects as parameters
    public static void displayHigherSalary(Employee emp1, Employee emp2) {
        System.out.println("--- Salary Comparison ---");
        System.out.println("Employee 1: " + emp1.getName() + " ($" + emp1.getSalary() + ")");
        System.out.println("Employee 2: " + emp2.getName() + " ($" + emp2.getSalary() + ")");
        System.out.println("-------------------------");

        // Compare the salaries using the getter methods
        if (emp1.getSalary() > emp2.getSalary()) {
            System.out.println("Result: " + emp1.getName() + " has the higher salary.");
        } else if (emp2.getSalary() > emp1.getSalary()) {
            System.out.println("Result: " + emp2.getName() + " has the higher salary.");
        } else {
            System.out.println("Result: Both employees earn the exact same salary.");
        }
    }

    public static void main(String[] args) {
        // 1. Create two Employee objects
        Employee employeeA = new Employee("David", 55000.0);
        Employee employeeB = new Employee("Sarah", 62000.0);

        // 2. Pass the objects to the comparison method
        displayHigherSalary(employeeA, employeeB);
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac employeeobjects.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java employeeobjects
--- Salary Comparison ---
Employee 1: David ($55000.0)
Employee 2: Sarah ($62000.0)
-------------------------
Result: Sarah has the higher salary.
```

Experiment 10: Returning Objects

Program 10.1: Returning a Student Object

Problem Statement:

Develop a Java program in which a method creates and returns a Student object. Display the returned object's details.

```
// Define the Student class
class Student {
    private int rollNumber;
    private String name;
    private String major;

    // Constructor to initialize the student details
    public Student(int rollNumber, String name, String major) {
        this.rollNumber = rollNumber;
        this.name = name;
        this.major = major;
    }

    // Method to display the student's details
    public void displayInfo() {
        System.out.println("Roll Number : " + rollNumber);
        System.out.println("Name        : " + name);
        System.out.println("Major       : " + major);
    }
}

// Main class renamed to 'Studentobjects'
public class Studentobjects {

    // Method that creates and returns a Student object
    public static Student createNewStudent() {
        System.out.println("Creating a new Student object inside the method...");

        // Instantiating a new Student object
        Student tempStudent = new Student(104, "Elena Gilbert", "Data Science");

        // Returning the object to the caller
        return tempStudent;
    }

    public static void main(String[] args) {
        System.out.println("=== Step 1: Call Method ===");
        // Call the method and store the returned object in a reference variable
        Student returnedStudent = createNewStudent();

        System.out.println("\n=== Step 2: Display Returned Object ===");
        // Call the display method using the returned object
        returnedStudent.displayInfo();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Studentobjects.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Studentobjects
=== Step 1: Call Method ===
Creating a new Student object inside the method...

=== Step 2: Display Returned Object ===
Roll Number : 104
Name        : Elena Gilbert
Major       : Data Science
```

Program 10.2: Returning a Bank Account Object

Problem Statement:

Write a Java program in which a method updates the balance of a BankAccount object and returns the updated object.

```java
// Define the BankAccount class
class BankAccount {
    private String accountNumber;
    private String accountHolder;
    private double balance;

    // Constructor to initialize the bank account
    public BankAccount(String accountNumber, String accountHolder, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = balance;
    }

    // Method to add money to the balance
    public void deposit(double amount) {
        this.balance += amount;
    }

    // Method to display account details
    public void displayAccountInfo() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Account Holder : " + accountHolder);
        System.out.println("Current Balance: $" + balance);
        System.out.println("--------------------------");
    }
}

// Main class renamed to 'bankaccountobject'
public class bankaccountobject {

    // Method that takes a BankAccount object, updates its balance, and returns it
    public static BankAccount addBonusAndReturn(BankAccount account, double bonusAmount) {
        System.out.println("Applying a bonus of $" + bonusAmount + " to the account...");

        // Update the state of the passed object
        account.deposit(bonusAmount);

        // Return the updated object
        return account;
    }

    public static void main(String[] args) {
        // 1. Create a BankAccount object
        BankAccount myAccount = new BankAccount("ACC-12345", "Michael Scott", 1500.00);

        System.out.println("=== Before Update ===");
        myAccount.displayAccountInfo();

        // 2. Pass the object to the method and catch the returned object
        BankAccount updatedAccount = addBonusAndReturn(myAccount, 500.00);

        System.out.println("\n=== After Update ===");
        // 3. Display details using the returned object reference
        updatedAccount.displayAccountInfo();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac bankaccountobject.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java bankaccountobject
=== Before Update ===
Account Number : ACC-12345
Account Holder : Michael Scott
Current Balance: $1500.0
--------------------------
Applying a bonus of $500.0 to the account...

=== After Update ===
Account Number : ACC-12345
Account Holder : Michael Scott
Current Balance: $2000.0
--------------------------
```

Experiment 11: Understanding static

Program 11.1: Static Variable

Problem Statement:

Develop a Java program to count the total number of Student objects created using a static variable.

```
// Define the Student class
class Student {
    // 1. Declare a static variable to keep track of the object count
    private static int studentCount = 0;

    private String name;
    private int rollNumber;

    // Constructor to initialize the student
    public Student(String name, int rollNumber) {
        this.name = name;
        this.rollNumber = rollNumber;

        // 2. Increment the static variable every time the constructor is called
        studentCount++;
        System.out.println("Created Student: " + this.name);
    }

    // Static method to retrieve the total count
    public static int getStudentCount() {
        return studentCount;
    }
}

// Main class to demonstrate the static counter
public class StudentCounterDemo {

    public static void main(String[] args) {
        System.out.println("Initial Student count: " + Student.getStudentCount());
        System.out.println("\n--- Creating Student Objects ---");

        // Create multiple Student objects
        Student student1 = new Student("Alice Johnson", 101);
        Student student2 = new Student("Bob Smith", 102);
        Student student3 = new Student("Charlie Brown", 103);
        Student student4 = new Student("Diana Prince", 104);

        System.out.println("\n--- Final Object Count ---");

        // 3. Access the static method to see the final count
        System.out.println("Total Student objects created: " + Student.getStudentCount());
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac StudentCounterDemo.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java StudentCounterDemo
Initial Student count: 0

--- Creating Student Objects ---
Created Student: Alice Johnson
Created Student: Bob Smith
Created Student: Charlie Brown
Created Student: Diana Prince

--- Final Object Count ---
Total Student objects created: 4
```

Program 11.2: Static Methods

Problem Statement:

Write a Java program to create a utility class containing static methods for calculating the square, cube, and factorial of a number.

```
class MathUtility {

        private MathUtility() {
        throw new UnsupportedOperationException("Utility class cannot be instantiated");
    }

        public static int square(int number) {
        return number * number;
    }

        public static int cube(int number) {
        return number * number * number;
    }


    public static long factorial(int number) {
        if (number < 0) {
            throw new IllegalArgumentException("Factorial is not defined for negative numbers.");
        }
        long result = 1;
        for (int i = 1; i <= number; i++) {
            result *= i;
        }
        return result;
    }
}

public class UtilityDemo {
    public static void main(String[] args) {
        System.out.println("=== Math Utility Calculations ===");

        int num1 = 5;
        int num2 = 4;
        int num3 = 6;


        int squareResult = MathUtility.square(num1);
        int cubeResult = MathUtility.cube(num2);
        long factorialResult = MathUtility.factorial(num3);

        System.out.println("The square of " + num1 + " is    : " + squareResult);
        System.out.println("The cube of " + num2 + " is      : " + cubeResult);
        System.out.println("The factorial of " + num3 + " is : " + factorialResult);
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac UtilityDemo.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java UtilityDemo
=== Math Utility Calculations ===
The square of 5 is    : 25
The cube of 4 is      : 64
The factorial of 6 is : 720
```

Experiment 12: Introducing final

Program 12.1: Final Keyword

Problem Statement:

Develop a Java program to demonstrate the use of final variables, final methods, and final classes.

```
final class SecureSystem {
    public void displayStatus() {
        System.out.println("SecureSystem is a final class. It cannot be subclassed.");
    }
}

class Parent {
    final int MAX_SPEED = 120;

    public final void showMaxSpeed() {
        System.out.println("The maximum speed is strictly set to: " + MAX_SPEED + " km/h");
    }
}

class Child extends Parent {
    public void childBehavior() {
        System.out.println("Inside the Child class.");
    }
}

public class FinalKeywordDemo {
    public static void main(String[] args) {
        System.out.println("=== 1. Final Variable and Method Demo ===");
        Child childObj = new Child();
        childObj.childBehavior();
        childObj.showMaxSpeed();

        System.out.println("\n=== 2. Final Class Demo ===");
        SecureSystem system = new SecureSystem();
        system.displayStatus();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac FinalKeywordDemo.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java FinalKeywordDemo
=== 1. Final Variable and Method Demo ===
Inside the Child class.
The maximum speed is strictly set to: 120 km/h

=== 2. Final Class Demo ===
SecureSystem is a final class. It cannot be subclassed.
```

Program 12.2: Blank Final Variable

Problem Statement:

Write a Java program to initialize a blank final variable through a constructor and use it to assign a unique employee ID.

```
class Employee {
    private static int idGenerator = 1000;
    private final int employeeId;
    private String name;

    public Employee(String name) {
        this.name = name;
        idGenerator++;
        this.employeeId = idGenerator;
        System.out.println("Employee created: " + this.name + " (ID Assigned: " + this.employeeId + ")");
    }

    public void displayDetails() {
        System.out.println("Employee ID: " + employeeId + " | Name: " + name);
    }
}

public class BlankFinalDemo {
    public static void main(String[] args) {
        System.out.println("=== Creating Employees ===");

        Employee emp1 = new Employee("Alice Johnson");
        Employee emp2 = new Employee("Bob Smith");
        Employee emp3 = new Employee("Charlie Brown");

        System.out.println("\n=== Employee Records ===");

        emp1.displayDetails();
        emp2.displayDetails();
        emp3.displayDetails();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac BlankFinalDemo.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java BlankFinalDemo
=== Creating Employees ===
Employee created: Alice Johnson (ID Assigned: 1001)
Employee created: Bob Smith (ID Assigned: 1002)
Employee created: Charlie Brown (ID Assigned: 1003)

=== Employee Records ===
Employee ID: 1001 | Name: Alice Johnson
Employee ID: 1002 | Name: Bob Smith
Employee ID: 1003 | Name: Charlie Brown
```

Program 13.1: College and Department

Problem Statement:

Develop a Java program to create a College class containing a static nested Department class. Display the department details.

```
class College {

    static class Department {
        String deptName;
        int deptCode;

        public Department(String deptName, int deptCode) {
            this.deptName = deptName;
            this.deptCode = deptCode;
        }

        public void displayDetails() {
            System.out.println("Department Name: " + deptName);
            System.out.println("Department Code: " + deptCode);
        }
    }
}

public class CollegeDemo {
    public static void main(String[] args) {
        College.Department dept1 = new College.Department("Computer Science", 101);
        College.Department dept2 = new College.Department("Electrical Engineering", 102);

        dept1.displayDetails();
        System.out.println("------------------------");
        dept2.displayDetails();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac CollegeDemo.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java CollegeDemo
Department Name: Computer Science
Department Code: 101
------------------------
Department Name: Electrical Engineering
Department Code: 102
```

Program 13.2: Employee Address

Problem Statement:

Write a Java program to create an Employee class with a static nested Address class and demonstrate accessing the nested class from the main method.

```
class Employee {
    String name;
    int employeeId;
    Address address;

    public Employee(String name, int employeeId, Address address) {
        this.name = name;
        this.employeeId = employeeId;
        this.address = address;
    }

    public void displayEmployeeDetails() {
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Name: " + name);
        address.displayAddress();
    }

    static class Address {
        String street;
        String city;
        String zipCode;

        public Address(String street, String city, String zipCode) {
            this.street = street;
            this.city = city;
            this.zipCode = zipCode;
        }

        public void displayAddress() {
            System.out.println("Address: " + street + ", " + city + " - " + zipCode);
        }
    }
}

public class EmployeeDemo {
    public static void main(String[] args) {
        Employee.Address empAddress = new Employee.Address("123 Tech Lane", "Silicon Valley", "94043");

        Employee emp = new Employee("John Doe", 1001, empAddress);

        emp.displayEmployeeDetails();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac EmployeeDemo.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java EmployeeDemo
Employee ID: 1001
Name: John Doe
Address: 123 Tech Lane, Silicon Valley - 94043
```

Experiment 14: Inner Classes

Program 14.1: Student Address

Problem Statement:

Develop a Java program to create a Student class containing an inner Address class. Display the student and address information.

```
class Student {
    String name;
    int rollNumber;

    public Student(String name, int rollNumber) {
        this.name = name;
        this.rollNumber = rollNumber;
    }

    public void displayStudentInfo() {
        System.out.println("Student Name: " + name);
        System.out.println("Roll Number: " + rollNumber);
    }

    class Address {
        String street;
        String city;
        String zipCode;

        public Address(String street, String city, String zipCode) {
            this.street = street;
            this.city = city;
            this.zipCode = zipCode;
        }

        public void displayAddressInfo() {
            System.out.println("Address: " + street + ", " + city + " - " + zipCode);
        }
    }
}

public class StudentDemo {
    public static void main(String[] args) {
        Student student = new Student("Alice Smith", 101);

        Student.Address address = student.new Address("456 University Ave", "Boston", "02115");

        student.displayStudentInfo();
        address.displayAddressInfo();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac StudentDemo.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java StudentDemo
Student Name: Alice Smith
Roll Number: 101
Address: 456 University Ave, Boston - 02115
```

Program 14.2: Library Management

Problem Statement:

Write a Java program to create a Library class containing an inner Book class. Demonstrate how the inner class accesses the members of the outer class.

```
class Library {
    private String libraryName;

    public Library(String libraryName) {
        this.libraryName = libraryName;
    }

    class Book {
        private String title;

        public Book(String title) {
            this.title = title;
        }

        public void displayBookDetails() {
            System.out.println("Book Title: " + title);
            System.out.println("Available at: " + libraryName);
        }
    }
}

public class LibraryDemo {
    public static void main(String[] args) {
        Library library = new Library("City Central Library");

        Library.Book book1 = library.new Book("The Great Gatsby");
        Library.Book book2 = library.new Book("1984");

        book1.displayBookDetails();
        System.out.println("-------------------------");
        book2.displayBookDetails();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac LibraryDemo.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java LibraryDemo
Book Title: The Great Gatsby
Available at: City Central Library
-------------------------
Book Title: 1984
Available at: City Central Library
```